Assignment: Abstract Classes and Final Methods

Objective

To understand and implement abstract classes, abstract methods, and final methods in Java, and to observe how polymorphism works with abstract references.

Problem Statement

You are required to design and implement Java programs using abstract classes to model real-world scenarios.

This assignment focuses on:

- Abstract classes and abstract methods
- Final methods and their restrictions
- Polymorphism using abstract references

Question 1: Online Payment System

Problem Statement

Design an **online payment system** using an abstract class.

Abstract Class: Payment

Attributes

- paymentId
- amount
- payerName
- status (possible values: `"PENDING"`, `"SUCCESS"`, `"FAILED"`)

Methods in Payment

1. `void printSummary()`
    - Prints all payment details
2. `final void process()`
    - Defines the complete payment workflow:
        - Step 1: Call `validate()`
        - Step 2: Call `deductAmount()`
        - Step 3: Call `sendNotification()`
        - Step 4: Set status as `"SUCCESS"` if all steps pass
    - **This method must NOT be overridden**

3. **Abstract Methods** (must be implemented by subclasses):
   - `abstract boolean validate()`
   - `abstract void deductAmount()`
   - `abstract void sendNotification()`

Subclasses of Payment

A) CardPayment

Additional Attributes

- cardNumber
- cvv

Validation Rules

- Card number must be exactly 16 digits
- CVV must be exactly 3 digits
- Amount must be greater than 0

After Successful Validation

- Print message for amount deduction
- Print message for notification

B) UPIPayment

Additional Attributes

- upiId

Validation Rules

- UPI ID must contain the `@` symbol
- Amount must be between 1 and 100000

After Successful Validation

- Print message for amount deduction
- Print message for notification

Main Method Requirements

- Create **2–4 Payment objects** (`CardPayment` / `UPIPayment`)
- Call the `process()` method on each object
- Call `printSummary()` to display final payment details

FirstBitSolutions.com ...Learn IT Bit by Bit...

Question 2: Electricity Bill Calculator

Problem Statement

Create a program to calculate electricity bills for **Residential** and **Commercial** customers using abstract classes.

Abstract Class: ElectricityBill

Attributes

- customerName
- units (units consumed)

Methods

1. `void showUsage()`
   - Prints customer name and units consumed
2. `abstract double calculateBill()`
   - Each customer type must implement its own bill calculation logic
3. `final void generateBill()`
   - Performs the following steps:
     - Call `calculateBill()`
     - Add 5% tax
     - Add fixed charge of ₹50
     - Print the final bill amount
   - **This method must NOT be overridden**

Subclass: ResidentialBill

Billing Slabs

- 0–100 units → ₹2.5 per unit
- 101–300 units → ₹3.5 per unit
- Above 300 units → ₹5 per unit

Additional Condition

- If units > 500 → Add surcharge of ₹150

Subclass: CommercialBill

Charges

- ₹6.5 per unit

FirstBitSolutions.com ...Learn IT Bit by Bit...

Conditions

1. If units < 200 → Minimum bill = ₹1500
2. If units > 1000 → Add 8% energy surcharge
    - (Apply surcharge only on unit charges, not on fixed charges)

Main Method Requirements

- Ask user to select customer type:
    - Residential
    - Commercial
- Input customer name and units consumed
- Create the appropriate object
- Call `generateBill()` to display the final bill

Question 3: Insurance Premium Calculator

Problem Statement

Design an insurance premium calculator using abstract classes for different policy types.

Abstract Class: InsurancePolicy

Attributes

- policyHolderName
- basePremium

Methods

1. `abstract double calculatePremium()`
    - Each policy type must implement its own premium calculation logic
2. `void printPolicyDetails()`
    - Prints:
        - Policy holder name
        - Base premium
        - Final premium (using `calculatePremium()`)

Subclass: CarInsurance

Attributes

- carAgeInYears
- hadAccidentInLastYear (true / false)
- carValue (in ₹)

FirstBitSolutions.com ...Learn IT Bit by Bit...

Premium Calculation Rules

- Car age ≤ 3 years → Add 10%
- Car age 4–7 years → Add 20%
- Car age > 7 years → Add 30%
- Accident in last year → Add 25% loading
- No accident → Apply 10% discount
- Car value > ₹10,00,000 → Add ₹2000 extra

Subclass: HealthInsurance

Attributes

- age
- isSmoker (true / false)
- hasPreExistingDisease (true / false)

Premium Calculation Rules

- Age < 30 → Add 10%
- Age 30–45 → Add 25%
- Age > 45 → Add 40%
- Smoker → Add 30% loading
- Non-smoker → Apply 5% discount
- Pre-existing disease → Add 20%

Main Method Requirements

- Ask user to choose policy type:
    1. Car Insurance
    2. Health Insurance
- Input all required details
- Create the appropriate object
- Call `printPolicyDetails()` to display final premium

Important Rules

• Do not modify the given structure
• Use abstract methods exactly as specified
• Do not override final methods
• Follow proper Java naming conventions
• Write clean, readable, and modular code

FirstBitSolutions.com
...Learn IT Bit by Bit...